

Automated Data load Validation in Snowflake

--Step:-1-----Create Database and schema-----

```
CREATE OR REPLACE DATABASE AUTO_LOAD_DB;
```

```
CREATE OR REPLACE SCHEMA AUTO_LOAD_DB.PUBLIC;
```

```
USE DATABASE AUTO_LOAD_DB;
```

```
USE SCHEMA PUBLIC;
```

--Step:- 2-----Create Final Target Table----- (This table stores only clean/validated data)

```
CREATE OR REPLACE TABLE CUSTOMER_TARGET (  
    CUSTOMER_ID INT,  
    CUSTOMER_NAME STRING,  
    EMAIL STRING,  
    CITY STRING,  
    CREATED_AT DATE  
);
```

--Step:- 3-----Create Audit Log Table_____ (This table stores success/failure details)

```
CREATE OR REPLACE TABLE LOAD_AUDIT_LOG (  
    LOG_ID INT AUTOINCREMENT,  
    FILE_NAME STRING,  
    LOAD_STATUS STRING,  
    ROW_COUNT INT,  
    ERROR_MESSAGE STRING,  
    LOAD_TIME TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()  
);
```

Step:-4-----Create CSV File Format-----

```
CREATE OR REPLACE FILE FORMAT CSV_FILE_FORMAT  
TYPE = CSV  
FIELD_DELIMITER = ','  
SKIP_HEADER = 1  
FIELD_OPTIONALLY_ENCLOSED_BY = ''"  
NULL_IF = ('NULL', 'null', '')  
EMPTY_FIELD_AS_NULL = TRUE;
```

---Step 5: Create Snowflake Internal Stage-----

```
CREATE OR REPLACE STAGE CUSTOMER_STAGE  
FILE_FORMAT = CSV_FILE_FORMAT;
```

```
LIST @CUSTOMER_STAGE;
```

-----Step 7: Create Stored Procedure-----

```
CREATE OR REPLACE PROCEDURE LOAD_VALIDATE_CUSTOMER(FILE_NAME STRING)  
RETURNS STRING
```

```
LANGUAGE SQL
```

```
AS
```

```
$$
```

```
DECLARE
```

```
    NULL_COUNT INT DEFAULT 0;
```

```
    DUP_COUNT INT DEFAULT 0;
```

```
    TOTAL_ROWS INT DEFAULT 0;
```

```
    RESULT_MSG STRING;
```

```
    COPY_SQL STRING;
```

```
BEGIN
```

```
    CREATE OR REPLACE TEMPORARY TABLE TEMP_CUSTOMER_LOAD (  
        CUSTOMER_ID INT,  
        CUSTOMER_NAME STRING,  
        EMAIL STRING,  
        CITY STRING,  
        CREATED_AT DATE  
    );
```

```
COPY_SQL :=
```

```
    'COPY INTO TEMP_CUSTOMER_LOAD  
    FROM @CUSTOMER_STAGE  
    FILES = (" || FILE_NAME || ")  
    FILE_FORMAT = (FORMAT_NAME = CSV_FILE_FORMAT)  
    ON_ERROR = "CONTINUE";
```

```
EXECUTE IMMEDIATE :COPY_SQL;
```

```
SELECT COUNT(*) INTO :TOTAL_ROWS  
FROM TEMP_CUSTOMER_LOAD;
```

```
SELECT COUNT(*) INTO :NULL_COUNT  
FROM TEMP_CUSTOMER_LOAD
```

```
WHERE CUSTOMER_ID IS NULL  
OR CUSTOMER_NAME IS NULL  
OR EMAIL IS NULL;
```

```
SELECT COUNT(*) INTO :DUP_COUNT  
FROM (  
    SELECT CUSTOMER_ID  
    FROM TEMP_CUSTOMER_LOAD  
    WHERE CUSTOMER_ID IS NOT NULL  
    GROUP BY CUSTOMER_ID  
    HAVING COUNT(*) > 1  
);
```

```
IF (:NULL_COUNT = 0 AND :DUP_COUNT = 0) THEN
```

```
    INSERT INTO CUSTOMER_TARGET  
    SELECT CUSTOMER_ID, CUSTOMER_NAME, EMAIL, CITY, CREATED_AT  
    FROM TEMP_CUSTOMER_LOAD;
```

```
    INSERT INTO LOAD_AUDIT_LOG  
    (  
        FILE_NAME,  
        LOAD_STATUS,  
        ROW_COUNT,  
        ERROR_MESSAGE  
    )  
    VALUES  
    (  
        :FILE_NAME,  
        'SUCCESS',  
        :TOTAL_ROWS,  
        NULL  
    );
```

```
    RESULT_MSG := 'SUCCESS: File loaded and validated successfully. Rows inserted: ' ||  
:TOTAL_ROWS;
```

```
ELSE
```

```
    INSERT INTO LOAD_AUDIT_LOG  
    (  
        FILE_NAME,  
        LOAD_STATUS,  
        ROW_COUNT,
```

```

        ERROR_MESSAGE
    )
VALUES
(
    :FILE_NAME,
    'FAILED',
    :TOTAL_ROWS,
    'Validation failed. Null count: ' || :NULL_COUNT || ', Duplicate count: ' || :DUP_COUNT
);

RESULT_MSG := 'FAILED: Validation failed. Null count: ' || :NULL_COUNT || ', Duplicate
count: ' || :DUP_COUNT;

END IF;

RETURN RESULT_MSG;

EXCEPTION
WHEN OTHER THEN

    INSERT INTO LOAD_AUDIT_LOG
    (
        FILE_NAME,
        LOAD_STATUS,
        ROW_COUNT,
        ERROR_MESSAGE
    )
VALUES
(
    :FILE_NAME,
    'FAILED',
    0,
    'Unexpected error occurred during load'
);

RETURN 'FAILED: Unexpected error occurred during load';

END;
$$;

LIST @CUSTOMER_STAGE;

```

-----Step 8: Test with Valid CSV File-----

```
CALL LOAD_VALIDATE_CUSTOMER('customer_valid_20_records.csv');
```

```
SELECT * FROM CUSTOMER_TARGET;
```

```
SELECT * FROM LOAD_AUDIT_LOG  
ORDER BY LOAD_TIME DESC;
```

-----Step 9: Test with Invalid CSV File (Now call the invalid file)-----

```
CALL LOAD_VALIDATE_CUSTOMER('customer_invalid_20_records.csv');
```

```
SELECT * FROM LOAD_AUDIT_LOG  
ORDER BY LOAD_TIME DESC;
```